

CS151 Intro to Data Structures

Maps

Announcements

Lab 8 and HW6 due Monday 4/6

Warm up: Choosing the Right Data Structure...

You are building a task management system that needs to handle various operations efficiently. Each task has an associated importance level, and tasks can be completed, added, or queried frequently. The operations that your system needs to support are:

1. **Add a task:** Insert a new task with a specified importance level.
2. **Remove most important task:** Complete and remove the task with the highest importance.
3. **Get most important task:** View the task with the highest importance without removing it.
4. **Check if tasks are available:** Determine if there are any tasks left to complete.
5. **Update the importance of an existing task:** Change the importance level of a specific task.

Warm up: Choosing the Right Data Structure...

Priority Queue

1. **Add a task:**
 - a. $O(\log n)$
2. **Remove most important task:**
 - a. $O(\log n)$
3. **Get most important task:**
 - a. $O(1)$
4. **Check if tasks are available:**
 - a. $O(1)$
5. **Update the importance of an existing task:**
 - a. $O(\log n)$ assuming you have the location of the given task

Map

- How can we implement a map?
 - Array !

Map.Entry Interface

- A (Key, Value) pair
- Keys and Values can be any reference type
- Methods:
 - `getKey()`
 - `getValue()`
 - `setValue(V val)`
- **Implementation:** `SimpleEntry`

ArrayMap

Let's implement a `Map` as an array of `SimpleEntry`s

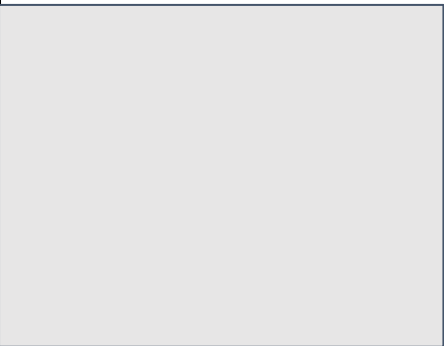
Performance Analysis

	Map implemented with underlying Array	LinkedList
get		
put		
remove		

LinkedList Map

- `get (K key)`
- `put (K key, V value)`
 - If `k` is not in the map add it. If it is in the map, replace with the new value.
- `remove (K key)`

Performance Analysis

	Array	LinkedList
get	$O(n)$	
put	$O(n)$	
remove	$O(n)$	

Hash Functions

Hash Functions

- A *hash function* maps an arbitrary length input to a fixed length *unique* output
- <https://emn178.github.io/online-tools/sha256.html>
- Applications
 - data structures
 - encryption / digital signatures
 - blockchain
- Properties of a good hash function:
 - one way
 - **collision resistant**
 - uniformity
 - quick to compute

A Simple Hash Function

Given an int x ...

$h(x)$ = last 4 digits of x

- one way?
- collision resistant?
- uniform?

Another Simple Hash Function

$$h(x) = x \% N$$

- one way?
- collision resistant?
- uniform?

HashMaps

- How can we use hash functions to improve the performance of our ArrayMap implementation?
- **Use hash as our array index!**
- How would the following operations look?
 - put
 - get
 - remove

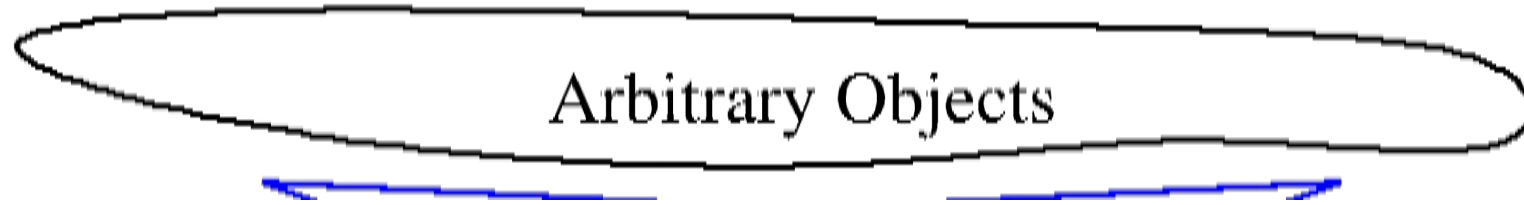
More on Hash Functions

A hash function is usually specified as the composition of two functions:

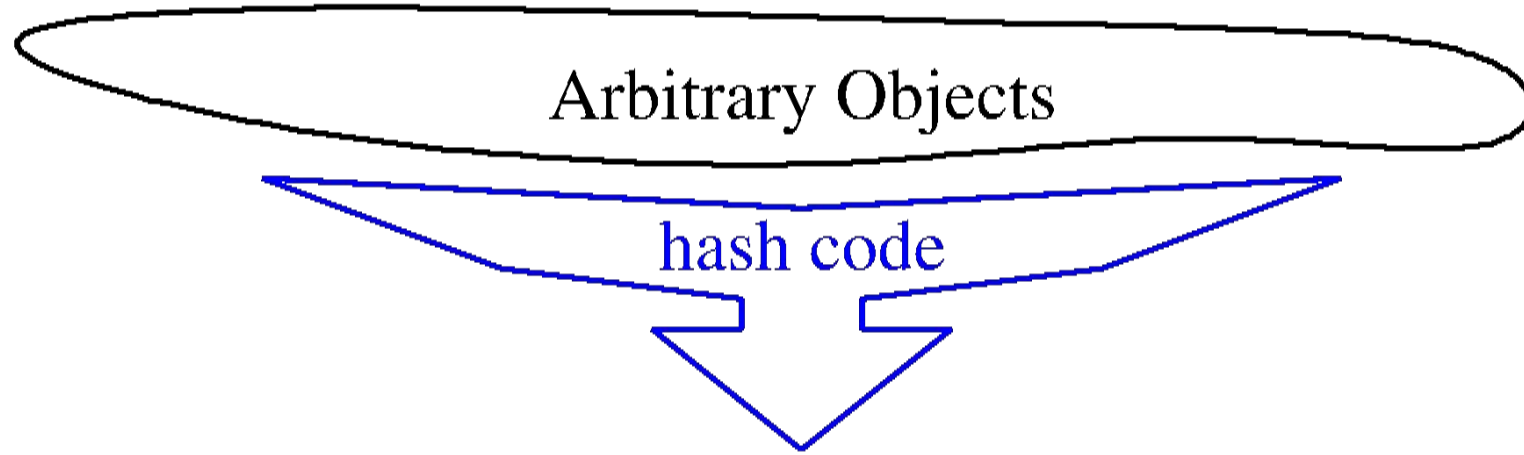
- hash code: $h_1: \text{key} \rightarrow \text{integers}$
- compression: $h_2: \text{integers} \rightarrow [0, N - 1]$
- $h(x) = h_2(h_1(x))$

The goal is to "disperse" the keys in an appropriately random way

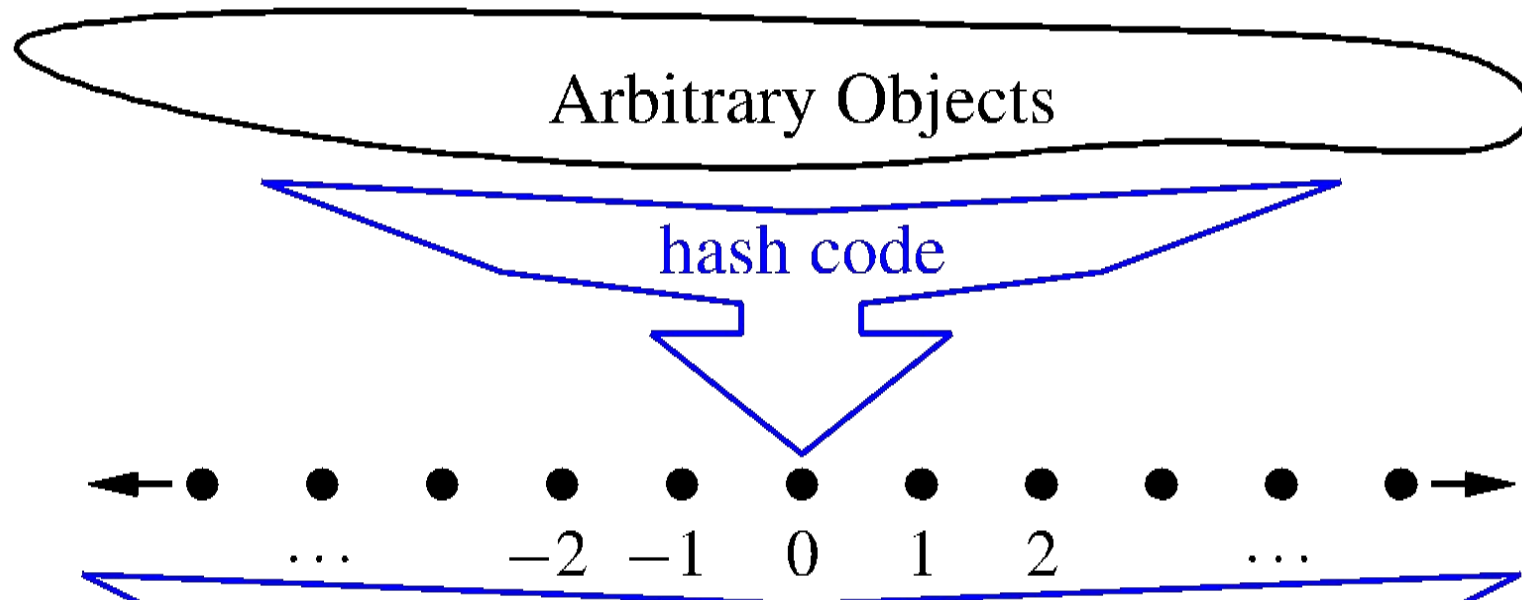
Hash Function Illustration



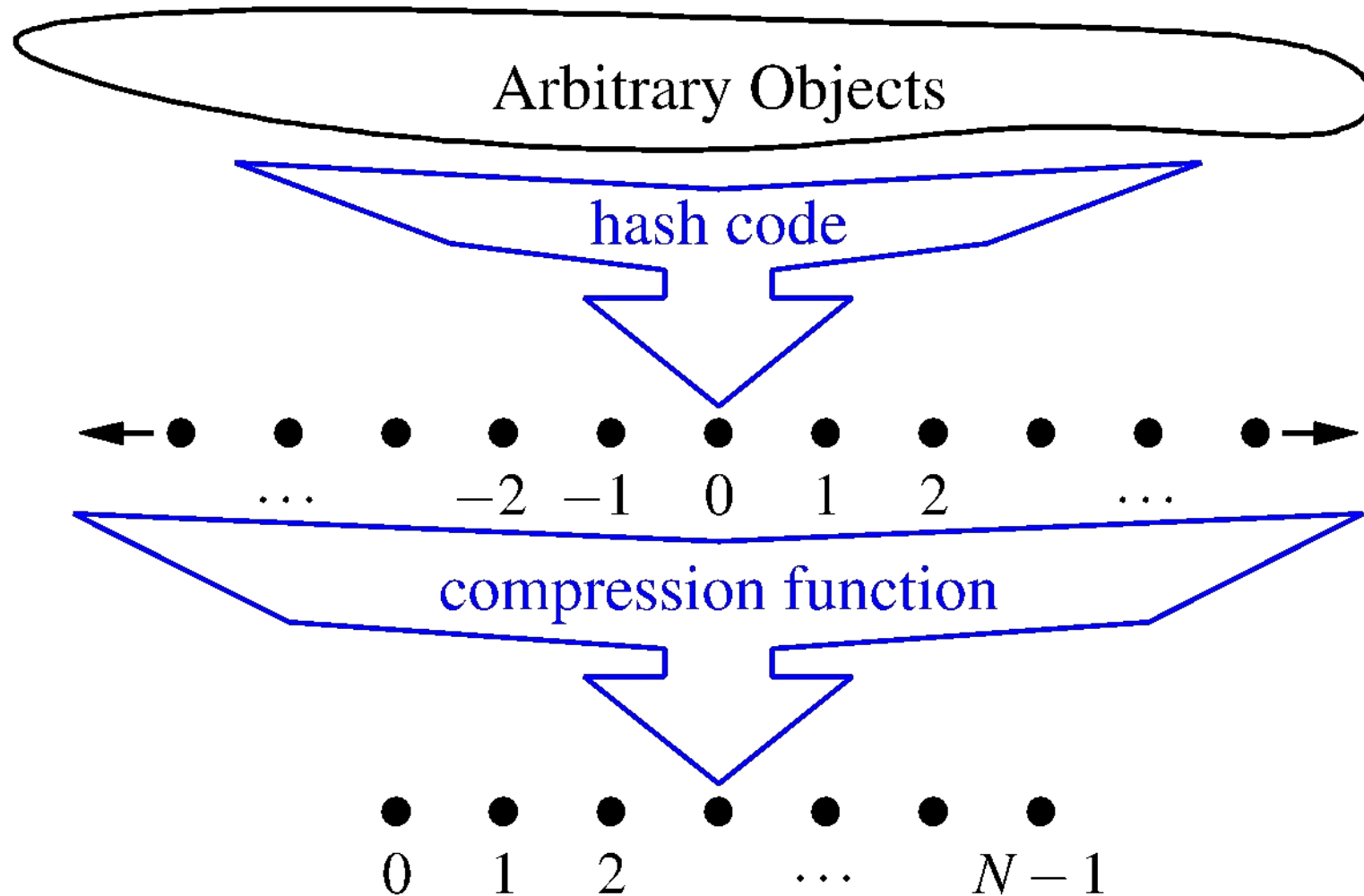
Hash Function Illustration



Hash Function Illustration



Hash Function Illustration



Hash Codes (h_1)

- $h_1(k)$ takes an arbitrary key and computes an integer
 - Goal: collision resistant!
 - Need not be a fixed length or in fixed range $[0, N)$
 - Can even be negative

Hash Codes h_1

Idea 1: Memory Addresses

- use the memory address where the keys are stored
- **default hash code for Java objects**

docs.oracle.com/javase/8/docs/api/java/lang/Object.html	
Method Summary	
All Methods	Instance Methods
Concrete Methods	
Modifier and Type	Method and Description
protected Object	clone() Creates and returns a copy of this object.
boolean	equals(Object obj) Indicates whether some other object is "equal to" this on
protected void	finalize() Called by the garbage collector on an object when garba object.
Class<?>	getClass() Returns the runtime class of this Object.
int	hashCode() Returns a hash code value for the object.

Hash Codes (h_1)

What if our key is not an object?

- Integer cast: `byte`, `short`, `int`, `char` and `float`
- What about `long` and `double`??
 - Can't cast to `int`. We'll lose information!
 - COLLISIONS
 - Instead, partition bits into `int` components and combine them

Compression (h_2)

Why do we need compression?

Compression Idea 1: mod

$$h_2(x) = x \bmod N$$

forces output to be in range $[0, N)$

How should we choose N ?

primes!

Compression (h_2)

Compression Idea 2: Multiply Add and Divide (MAD)

$$h_2(x) = ((ax + b) \bmod p) \bmod N$$

where N is the capacity

p is a prime $> N$

a and b are $[0, p)$

a scales the range

b shifts the start

HashMap

Book's `AbstractHashMap` hash method uses:

$$h_1(k) = k.\text{hashCode}() \text{ // java memory address}$$

$$h_2(x) = ((ax + b) \% p) \% N$$

Hash Maps

Efficient data structure that stores (Key, Value) pairs

Implements the Map ADT

- `get (k)` : if the map `M` has an entry with key `k`, return its associated value; else, return null
- `put (k, v)` : insert entry `(k, v)` into the map `M`; if key `k` is not already in `M`, then return null; else, replace old value with `v` and return old value associated with `k`
- `remove (k)` : if the map `M` has an entry with key `k`, remove it from `M` and return its associated value; else, return null
- `size ()`, `isEmpty ()`
- `keySet ()` : return an iterable collection of the keys in `M`
- `values ()` : return an iterator of the values in `M`
- `entrySet ()` : return an iterable collection of the entries in `M`

Hash Maps

Implementation

Let's start with our ArrayMap and use hashes for indices

BE CAREFUL! % MEANS REMAINDER IN JAVA NOT MOD!

What should we do if there's a collision?

- For a first impl, let's just overwrite

Performance Analysis

	ArrayMap	Collision Resistant Hash Map
get		
put		
remove		

AbstractHashMap

```
1 public abstract class AbstractHashMap<K,V> extends AbstractMap<K,V> {
2     protected int n = 0;           // number of entries in the dictionary
3     protected int capacity;        // length of the table
4     private int prime;              // prime factor
5     private long scale, shift;      // the shift and scaling factors
6     public AbstractHashMap(int cap, int p) {
7         prime = p;
8         capacity = cap;
9         Random rand = new Random();
10        scale = rand.nextInt(prime-1) + 1;
11        shift = rand.nextInt(prime);
12        createTable();
13    }
14    public AbstractHashMap(int cap) { this(cap, 109345121); } // default prime
15    public AbstractHashMap() { this(17); } // default capacity
16    // public methods
17    public int size() { return n; }
18    public V get(K key) { return bucketGet(hashValue(key), key); }
19    public V remove(K key) { return bucketRemove(hashValue(key), key); }
20    public V put(K key, V value) {
21        V answer = bucketPut(hashValue(key), key, value);
22        if (n > capacity / 2) // keep load factor <= 0.5
23            resize(2 * capacity - 1); // (or find a nearby prime)
24        return answer;
25    }
26 }
```

AbstractHashMap

```
26 // private utilities
27 private int hashCode(K key) {
28     return (int) ((Math.abs(key.hashCode())*scale + shift) % prime) % capacity);
29 }
30 private void resize(int newCap) {
31     ArrayList<Entry<K,V>> buffer = new ArrayList<>(n);
32     for (Entry<K,V> e : entrySet())
33         buffer.add(e);
34     capacity = newCap;
35     createTable(); // based on updated capacity
36     n = 0; // will be recomputed while reinserting entries
37     for (Entry<K,V> e : buffer)
38         put(e.getKey(), e.getValue());
39 }
40 // protected abstract methods to be implemented by subclasses
41 protected abstract void createTable();
42 protected abstract V bucketGet(int h, K k);
43 protected abstract V bucketPut(int h, K k, V v);
44 protected abstract V bucketRemove(int h, K k);
45 }
```

Handling Collisions

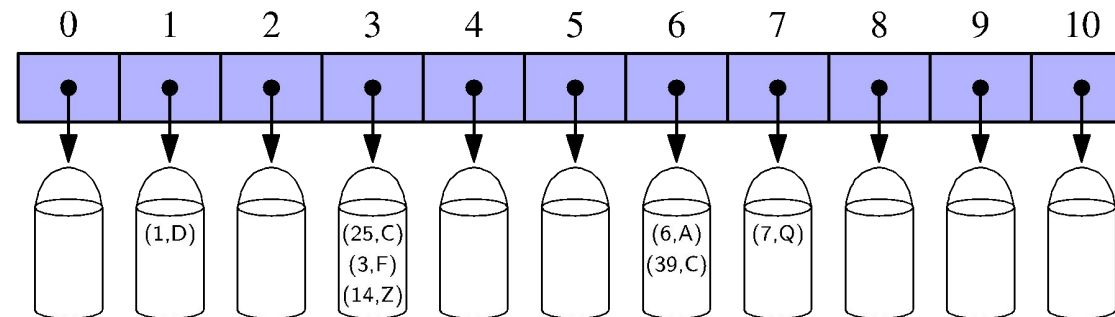
Handling Collisions

A hash function does not guarantee one-to-one mapping – no hash function does

One approach **chaining**:

When more than one key hash to the same index, we have a bucket

Each index holds a collection of entries



Collision Handling

Collisions occur when elements with different keys are mapped to the same cell

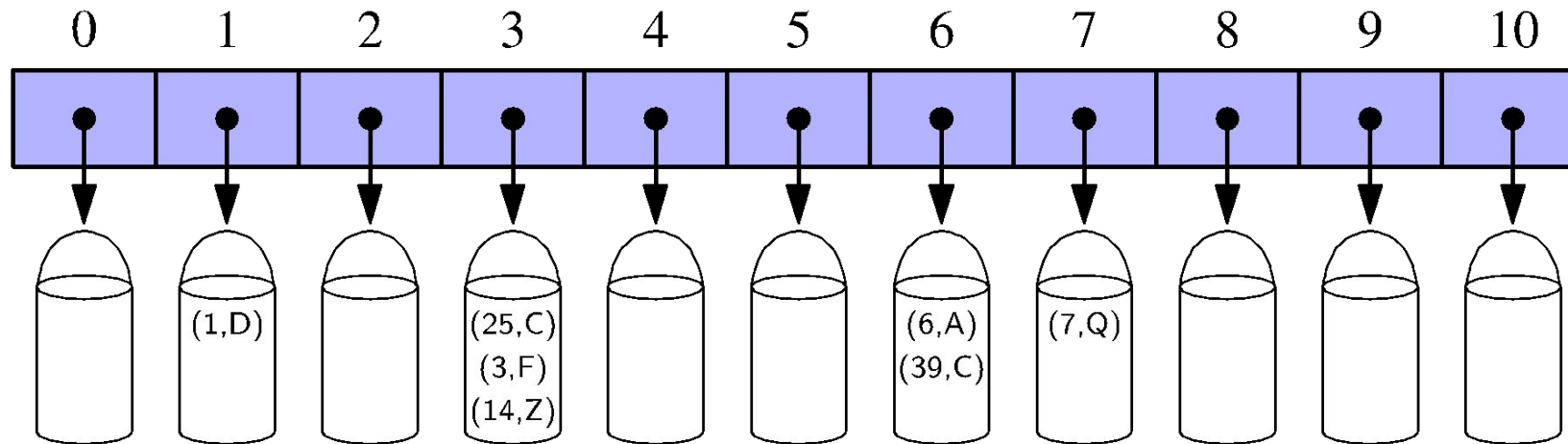
Separate Chaining: let each cell in the table point to a linked list of entries that map there

Simple, but requires additional memory besides the table

Let's implement a ChainHashMap

What data structure should we use for the buckets?

- LinkedList!



Summary

1. Maps
 - a. stores key-value pairs
2. Hash Functions
 - a. how can these be used to make more efficient maps?
 - b. Composition of h_1 and h_2
 - c. h_2 compresses output of h_1 between 0 and N
3. HashMap
 - a. Efficient data structure with constant time* access, insertion, and removal
 - b. * assuming no collisions or expansions